

University POLITEHNICA of Bucharest
Faculty of Automatic Control and Computers,
Computer Science and Engineering Department



Report 1

Run OpenArena Server on Unikraft

Scientific Adviser:
Razvan Deaconescu

Author:
Valentin-Razvan Bogdan

Bucharest, 2025

Contents

1	Introduction	1
1.1	Unikernels and Unikraft	1
2	Background	2
2.1	OpenArena	2
3	Motivation and Objectives	3
3.1	Motivation	3
3.2	Objectives	3
3.3	Use Cases	3
3.3.1	Main Use Case	3
3.3.2	Other Use Cases	3
4	State of the Art and Related Work	4
4.1	State of the Art	4
4.2	Related Work	4
4.3	Implementation Details	4
4.3.1	Running in Binary Compatible Mode	4
5	Future Work and Lessons Learned	5
5.1	Future Work	5
5.2	Lessons Learned	5

Chapter 1

Introduction

1.1 Unikernels and Unikraft

Unikernels² are specialized, single-address-space operating systems that offer benefits over well-known alternatives in terms of security, size, and performance. Their core principle lie in stripping away unnecessary components of the kernel, leaving only the essential code to run a specific application.

Often compared to containers and virtual machines, these promise better resource efficiency³ and security⁴ due to their minimalistic nature. Therefore, these results⁵ make unikernels an attractive choice for cloud computing and edge computing environments.

In this context, Unikraft⁶ emerges as an open-source community that leverages the potential of unikernels, facilitating this process by providing modular, reusable components that can be customized for specific use cases, ensuring that only the required libraries and functionalities are included, further optimizing the binary image.

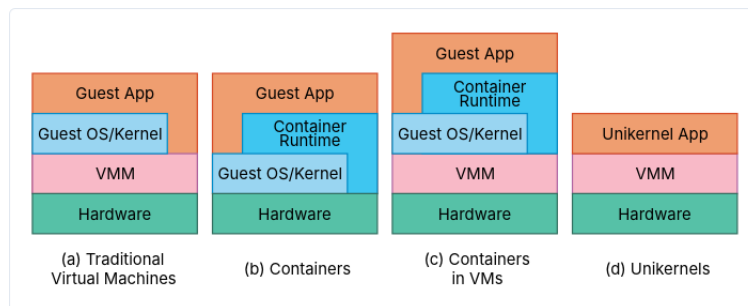


Figure 1.1: Methods for running applications⁷

²<https://wiki.xenproject.org/wiki/Unikernels>

³<https://unikraft.org/docs/concepts/efficiency>

⁴<https://unikraft.org/docs/concepts/security>

⁵<https://unikraft.org/docs/concepts/performance>

⁶<https://unikraft.org/>

Chapter 2

Background

2.1 OpenArena

OpenArena² is an open-source multiplayer first-person shooter game that was first released in 2005. It is written in C³ and is based on the Quake III Arena engine, originally developed by id Software. OpenArena retains the same core while offering a game with its unique characteristics and community-driven development.

A key feature of OpenArena is its client-server architecture, which allows players to host their own game servers, join multiplayer matches, and modify the game extensively. It supports game modifications (mods), allowing custom rulesets, new maps, and gameplay mechanics to be introduced.

The game assets, including textures, sounds and configurations, are packaged in .pk3⁴ files, making it easy for users to create and distribute custom content.

Despite being nearly two decades old, OpenArena remains actively played today, particularly within niche gaming communities and enthusiasts of classic arena shooters.



Figure 2.1: OpenArena gameplay

²<https://openarena.ws/smfnews.php>

³<https://github.com/OpenArena/engine>

⁴<https://doomwiki.org/wiki/PK3>

Chapter 3

Motivation and Objectives

3.1 Motivation

Currently, Unikraft lacks gaming-related projects. Therefore, integrating OpenArena extends its capabilities into a new domain: **gaming**.

Beyond demonstrating unikernels' applicability to non-traditional use cases, this project also enables a direct comparison between binary compatibility mode and native mode.

The resulting benchmarks will provide valuable insights into the trade-offs between ease of porting and performance gains.

3.2 Objectives

The key objectives of this thesis are:

- Run an OpenArena server from scratch (source files).
- Port the server to Unikraft in binary compatibility mode using the existing libelf library and a Dockerfile-based root filesystem.
- Port the server to Unikraft in native mode by creating an external library containing the required source files and dependencies.
- Perform benchmarks to compare the two approaches.

3.3 Use Cases

3.3.1 Main Use Case

Run an OpenArena server on Unikraft as well as allowing clients to connect and play.

3.3.2 Other Use Cases

Serve as a pioneer effort in bringing gaming applications to Unikraft, potentially leading to further developments in this area.

Chapter 4

State of the Art and Related Work

4.1 State of the Art

So far, the server has been successfully run locally from source code, inside a container using a Dockerfile, and ported to Unikraft in binary compatibility mode.

4.2 Related Work

Current efforts focus on understanding and implementing native execution for the server while analyzing similar projects that have tackled the same scenario.

4.3 Implementation Details

4.3.1 Running in Binary Compatible Mode

Two files play an essential role:

The **Dockerfile** is responsible for creating the unikernel filesystem. Initially, it compiles the OpenArena server from source as an ELF binary. Then, it constructs a minimal filesystem that includes only the necessary libraries while exposing a port to be used by the clients.

The **Kraftfile** configures and builds the unikernel, incorporating the Dockerfile and libraries such as libelf, libukboot, libvfscore and lwip.

For **building** and **running** the unikernel on x86 with KVM/QEMU, use:

```
1 kraft build --no-cache --no-update --plat qemu --arch x86_64 .
2 kraft run --rm -M 2048M -p 27960:27960/udp --plat qemu --arch x86_64 .
```

Listing 4.1: Build and run the OpenArena server

Chapter 5

Future Work and Lessons Learned

5.1 Future Work

The next step is to implement the native porting of the OpenArena server and wrap it into an external library in Unikraft.

Later on, we will compare the metrics resulted from the benchmarks and we will write down the most decisive differences.

5.2 Lessons Learned

- Using a shell inside a container can save hours of debugging an incorrectly specified filepath in a Dockerfile.